6.5.1

a

```
E.type = max(s.type, c.type) // → int
t1 = widen(s.addr, s.type, E.type) // → (int) s
t2 = widen(c.addr, c.type, E.type) // → (int) c
E.addr = new Temp()
gen(E.addr "=" t1 "+" t2)

x = widen(E.addr, E.type, x.type) // → (float) (t1 + t2)
```

b

```
E.type = max(s.type, c.type) // → int
t1 = widen(s.addr, s.type, E.type) // → (int) s
t2 = widen(c.addr, c.type, E.type) // → (int) c
E.addr = new Temp()
gen(E.addr "=" t1 "+" t2)

i = widen(E.addr, E.type, i.type) // → (int) (t1 + t2)
```

c

```
E1.type = max(s.type, c.type) // → int
t1 = widen(s.addr, s.type, E.type) // → (int) s
t2 = widen(c.addr, c.type, E.type) // → (int) c
E1.addr = new Temp()
gen(E1.addr "=" t1 "+" t2) // → E1 = (int) s + (int) c

E2.type = max(t.type, d.type) // → int
t3 = widen(t.addr, t.type, E2.type) // → (int) t
t4 = widen(d.addr, d.type, E2.type) // → (int) d
E2.addr = new Temp()
gen(E2.addr "=" t3 "+" t4) // → E2 = (int) t + (int) d

E3.type = max(E1.type, E2.type) // → int
t5 = widen(E1.addr, E1.type, E3.type) // → (int) E1
t6 = widen(E2.addr, E2.type, E3.type) // → (int) E2
E3.addr = new Temp()
```

```
gen(E3.addr "=" t5 "*" t6) // → E3 = (int) E1 * (int) E2

x = widen(E3.addr, E3.type, x.type) // → (float) E3
```

6.5.2

```
E.unique = (expected): { E2.unique(s) → t | s → t in E1.type, t in
expected }
```

Where `E.unique` is a lambda defined on `E` that takes in our expected type from higher-levels. Assuming we assign `E` from somewhere where we expect a type, like variable assignment. We call `E2.unique` which will return a static type and stop recursing if it is not a function.

7.2.3

a

```
f(5)
  |
  +- f(4)
  |    |
  |    +- f(3)
  |    |    |
  |    |    +- f(2)
  |    |    |    |
  |    |    |    +- f(1)
  |    |    |    +- f(0)
  |    |    +- f(1)
  |    +- f(2)
  |         |
  |         +- f(1)
  |         +- f(0)
  +- f(3)
       |
       +- f(2)
       |    |
       |    +- f(1)
       |    +- f(0)
       +- f(1)
```

b

```
f(5) (return = ?, int n = 5, local s = ?, local t = ?)
f(4) (return = ?, int n = 4, local s = ?, local t = ?)
f(3) (return = ?, int n = 3, local s = ?, local t = ?)
f(2) (return = ?, int n = 2, local s = ?, local t = ?)
f(1) (return = 1, int n = 1, local s = ?, local t = ?)
```

c

```
f(5) (return = ?, int n = 5, local s = 5, local t = 3)
f(4) (return = ?, int n = 4, local s = 3, local t = 2)
f(3) (return = ?, int n = 3, local s = 2, local t = 1)
f(2) (return = ?, int n = 2, local s = 1, local t = 1)
f(1) (return = 1, int n = 1, local s = ?, local t = ?)
```

7.4.1

```
80, 30, 60, 50, 70, 20, 40

First fit:
32 → 48 (32) 30 60 50 70 20 40
64 → 48 (32) 30 60 50 (64 + 6) 20 40
48 → (48) (32) 30 60 50 (64 + 6) 20 40
16 → (48) (32) 14 (16) 60 50 (64 + 6) 20 40

Best fit:
32 → 80 30 60 50 70 20 8 (32)
64 → 80 30 60 50 8 (64) 20 8 (32)
48 → 80 30 60 (48 + 2) 8 (64) 20 8 (32)
16 → 80 30 60 (48 + 2) 8 (64) 8 (16) 8 (32)
```

7.6.7

a

```
Unreached: A, B, C, D, E, F, G, H, I

Unscanned: A
Reached: A
Unscanned: C
Reached: A, C
Unscanned: F
```

```
Reached: A, C, F
Unscanned: H
Reached: A, C, F, H
Unscanned: I
Reached: A, C, F, H, I
Unscanned: G
Reached: A, C, F, H, I, G
Unscanned: E, H
Reached: A, C, F, H, I, G, E
Unscanned: H, C
Unscanned: C
Unscanned:

Unreached: B, D
```

b

```
Unreached: A, B, C, D, E, F, G, H, I

Unscanned: A
Reached: A
Unscanned: B, C
Reached: A, B
Unscanned: C, D, E
Reached: A, B, C
Unscanned: D, E, F
Reached: A, B, C, D
Unscanned: E, F, G
Reached: A, B, C, D, E
Unscanned: F, G, C
Reached: A, B, C, D, E, F
Unscanned: G, C, H
Reached: A, B, C, D, E, F, G
Unscanned: C, H, E, H
Unscanned: H, E, H
Reached: A, B, C, D, E, F, G, H
Unscanned: E, H, I
Unscanned: H, I
Unscanned: I
Reached: A, B, C, D, E, F, G, H, I
Unscanned:
```

```
Unreached:
```